

Topics Covered (Deep Explanation)

#	Topic	Page Focus
T1	IP Addressing – Binary, Classes, Special Addresses	Theory + Binary Examples
T2	Subnetting – Complete Step-by-Step (2 Numericals)	Class C + Class B Solved
T3	Distance Vector Routing – Full Table Iteration	Bellman-Ford Complete Example
T4	Link State Routing – Dijkstra Step-by-Step	Full Graph + All Steps
T5	ICMP – Every Message Type + Real Scenarios	ping + traceroute deep dive
T6	Congestion Control – Leaky + Token Bucket	Both Algorithms + Numericals
T7	ARP + RARP + DHCP – Complete Working	Packet by Packet Flow
T8	IP Fragmentation – Complete Numerical	Header Fields Explained
T9	IPv4 vs IPv6 + Transition Mechanisms	All Differences + Address Types

Topic 1 – IP Addressing

Deep Theory + Binary + Classes +
Special Addresses

IP Address Structure:

Ek IPv4 address **32 bits** ka hota hai — 4 octets mein likhte hain, dot se separate karke (dotted-decimal notation). Har octet = 8 bits = 0 to 255 value.

Example: 192.168.1.100 Binary: 11000000.10101000.00000001.01100100 IP Address =
Network ID (identifies the network) + Host ID (identifies specific device on that network)

Classful IP Addressing – 5 Classes

Class	Leading Bits	1st Octet	Net Bits	Host Bits	Subnets	Hosts/Net	Purpose
A	0xxxxxxx	1–126	/8	24 bits	126	16,777,214	Large ISPs, Govt
B	10xxxxxx	128–191	/16	16 bits	16,384	65,534	Universities, Companies
C	110xxxxx	192–223	/24	8 bits	2M+	254	Small businesses
D	1110xxxx	224–239	N/A	N/A	N/A	N/A	Multicast groups
E	1111xxxx	240–255	N/A	N/A	N/A	N/A	Research/Reserved

How to identify class quickly:

First octet check: 1–126 → Class A (127 = loopback, excluded) 128–191 → Class B
192–223 → Class C 224–239 → Class D (Multicast) 240–255 → Class E (Reserved)

Special & Private IP Addresses

Address / Range	Purpose	Routable on Internet?
127.0.0.0/8 (127.0.0.1)	Loopback – self-test; ping 127.0.0.1 = pings self	No
0.0.0.0	This host on this network (DHCP discovery source)	No
255.255.255.255	Limited broadcast – all hosts on current network	No
169.254.0.0/16	APIPA – auto-assigned when DHCP fails (link-local)	No
10.0.0.0/8	Private Class A – 16M hosts	No (use NAT)
172.16.0.0–172.31.255.255	Private Class B range	No (use NAT)
192.168.0.0/16	Private Class C – most home routers use this	No (use NAT)
224.0.0.0–239.255.255.255	Class D – Multicast (e.g. OSPF uses 224.0.0.5)	Partially

■ Formula

Hosts formula: Usable Hosts = $2^h - 2$ Subtract 2: (1) Network Address = all host bits 0 | (2) Broadcast = all host bits 1
Class A: $2^{24} - 2 = 16,777,214$ hosts
Class B: $2^{16} - 2 = 65,534$ hosts
Class C: $2^8 - 2 = 254$ hosts

Topic 2 – Subnetting

Deep Step-by-Step + Class C +
Class B Numericals

Subnetting kyon zaroori hai?

Ek company ko ek Class B network diya gaya jisme 65,534 hosts ho sakte hain. Lekin company ke sirf 300 employees hain — baaki 65,000+ addresses waste. Subnetting se network ko chhote logical segments mein divide karte hain: har department ka alag subnet → better security, less broadcast traffic, efficient use.

Subnetting – Complete Framework

1

Step 1: Identify class

First octet dekho → Class determine karo → Default mask pata chale

2

Step 2: Required subnets

Given subnets $\geq n \rightarrow 2^s \geq n \rightarrow$ Find minimum s (subnet bits)

3

Step 3: New mask

Default mask + borrow s bits = new subnet mask

4

Step 4: Block size

Block size = 2^h (h = remaining host bits) = 256 - last octet of mask

5

Step 5: Subnet table

Network addr starts at 0, increment by block size for each subnet

6

Step 6: Per subnet

Network = .0 of block; First host = .1; Last host = Broadcast-1; Broadcast = next block - 1

Numerical 1 – Class C: 200.1.2.0 → 5 subnets ■ PYQ

Given: Network = 200.1.2.0/24 (Class C) **Required:** Minimum 5 subnets ■■■ Step 1: Default info ■■■ Class C, Default mask = 255.255.255.0, Host octet = 4th octet ■■■ Step 2: Subnet bits ■■■ $2^s \geq 5 \rightarrow 2^1=2 \times 2^2=4 \times 2^3=8 \checkmark \rightarrow s = 3$ bits borrowed ■■■ Step 3: New subnet mask ■■■ Default: 11111111.11111111.11111111.00000000 = 255.255.255.0 Borrow 3: 11111111.11111111.11111111.11100000 = 255.255.255.224 = /27 ■■■ Step 4: Block size ■■■ 256 - 224 = 32 OR $2^h = 2^5 = 32$ Host bits remaining h = 8-3 = 5 → Hosts per subnet = $2^h - 2 = 30$ ■■■ Step 5: Total subnets = $2^3 = 8$ (need 5, so first 5 used) ■■■

Subnet	Network Addr	First Host	Last Host	Broadcast Addr	Usable Hosts	Mask
--------	--------------	------------	-----------	----------------	--------------	------

1 ✓	200.1.2.0	200.1.2.1	200.1.2.30	200.1.2.31	30	/27
2 ✓	200.1.2.32	200.1.2.33	200.1.2.62	200.1.2.63	30	/27
3 ✓	200.1.2.64	200.1.2.65	200.1.2.94	200.1.2.95	30	/27
4 ✓	200.1.2.96	200.1.2.97	200.1.2.126	200.1.2.127	30	/27
5 ✓	200.1.2.128	200.1.2.129	200.1.2.158	200.1.2.159	30	/27
6	200.1.2.160	200.1.2.161	200.1.2.190	200.1.2.191	30	/27
7	200.1.2.192	200.1.2.193	200.1.2.222	200.1.2.223	30	/27
8	200.1.2.224	200.1.2.225	200.1.2.254	200.1.2.255	30	/27

Numerical 2 – Class B: 172.16.0.0 → 12 subnets

Given: 172.16.0.0 (Class B, /16 = 255.255.0.0), Need 12 subnets
Step 1: $2^{\text{bits}} \geq 12 \rightarrow 2^4 = 16 \geq 12 \checkmark \rightarrow$
s=4 bits borrowed from 3rd octet
Step 2: New mask: 255.255.0.0 → borrow 4 bits from 3rd octet
3rd octet: 00000000 → 11110000 = 240
New mask = 255.255.240.0 = /20
Step 3: Block size = 256-240 = 16 (3rd octet increments by 16)
Host bits h = 16-4 = 12 → Hosts per subnet = $2^{12}-2 = 4094$
First 4 subnets (block of 16 in 3rd octet):
Subnet 1: 172.16.0.0 – 172.16.15.255 (hosts .0.1 to .15.254)
Subnet 2: 172.16.16.0 – 172.16.31.255
Subnet 3: 172.16.32.0 – 172.16.47.255
Subnet 4: 172.16.48.0 – 172.16.63.255 ... (total $2^4=16$ subnets possible, need 12 ✓)

CIDR – Classless Inter-Domain Routing

CIDR classful boundaries ko ignore karta hai. Mask ko **prefix-length** se denote karte hain — count of 1-bits in mask.

CIDR Notation	Subnet Mask	Host Bits	Max Hosts	Block Size
/24	255.255.255.0	8	254	256
/25	255.255.255.128	7	126	128
/26	255.255.255.192	6	62	64
/27	255.255.255.224	5	30	32
/28	255.255.255.240	4	14	16
/29	255.255.255.248	3	6	8
/30	255.255.255.252	2	2	4
/16	255.255.0.0	16	65534	65536
/20	255.255.240.0	12	4094	4096
/8	255.0.0.0	24	16777214	16777216

Topic 3 – Distance Vector Routing (DVR/RIP)

Bellman-Ford + Full Table Iterations
+ Count-to-Infinity

DVR – Core Concept:

DVR mein har router sirf apne **neighbors ke saath directly connected** hota hai. Woh apna routing table neighbors ko share karta hai, aur neighbors apna share karte hain. Aise karte karte poore network ki information phailti hai — "**routing by rumor**" bhi kehte hain.

Bellman-Ford Equation:

$D(x, y) = \min \text{ over all neighbors } v \{ \text{cost}(x, v) + D(v, y) \}$ where: $D(x, y)$ = minimum distance from router x to destination y $\text{cost}(x, v)$ = direct link cost from x to neighbor v $D(v, y)$ = v 's known distance to y

Full DVR Example – Network with 5 Routers

Network Topology: A --- 2 --- B --- 3 --- C --- 1 --- D --- 5 --- E
Direct link costs: A-B=2, A-D=7, B-C=3, B-D=4, C-D=1, D-E=5

Iteration 0 – Initialization (each router knows only direct links):

Router	To A	To B	To C	To D	To E
A	0	2	∞	7	∞
B	2	0	3	4	∞
C	∞	3	0	1	∞
D	7	4	1	0	5
E	∞	∞	∞	5	0

Iteration 1 – After 1st exchange (each router gets neighbors' tables):

Focus on Router A updates: A to C: via B = $\text{cost}(A,B) + D(B,C) = 2+3 = 5$ [was ∞ , update!]
A to D: via B = $2+4=6 < 7$ (direct) $\rightarrow$ update to 6 via B
A to E: via D = $7+5=12$ [was ∞ , update!]
Focus on Router E updates: E to A: via D = $5+7=12$ [was ∞]
E to B: via D = $5+4=9$ [was ∞]
E to C: via D = $5+1=6$ [was ∞]

Router	To A	To B	To C	To D	To E	Changed?
A	0	2	5 (via B)	6 (via B)	12 (via D)	Yes ✓
B	2	0	3	4	9 (via D)	Yes ✓
C	5 (via B)	3	0	1	6 (via D)	Yes ✓
D	6 (via B)	4	1	0	5	Yes ✓
E	12 (via D)	9 (via D)	6 (via D)	5	0	Yes ✓

Iteration 2 – After 2nd exchange (further improvements):

A to E: via B→D = 2+4+5=11 < 12 → update! E to A: via D→B→A = 5+4+2=11 < 12 → update! B to E: via D = 4+5=9 (already), via C→D = 3+1+5=9 (same)

Router	To A	To B	To C	To D	To E	Changed?
A	0	2	5	6	11 (via B)	Yes ✓
B	2	0	3	4	9	No
C	5	3	0	1	6	No
D	6	4	1	0	5	No
E	11 (via D→B)	9	6	5	0	Yes ✓

✓ Converged Result

Convergence: After iteration 3, no more changes → DVR has converged. **Final Shortest Paths from A:**
A→B = 2 (direct) A→C = 5 (via B) A→D = 6 (via B→D, NOT direct 7!) A→E = 11 (via B→D→E)

Count-to-Infinity Problem – Deep Explanation

■■ Count-to-Infinity Problem

Scenario: A ■■■1■■■ B ■■■1■■■ C Normal state: B knows A via direct (cost 1), C knows A via B (cost 2)

A-B link FAILS: B: "A via direct? FAILED!" → $\text{dist}(B,A) = \infty$ But BEFORE B updates, C tells B: "I can reach A in cost 2" B thinks: $C \rightarrow A = 2$, plus $B \rightarrow C = 1 \rightarrow \text{dist}(B,A) = 3 \leftarrow \text{WRONG! (A is unreachable)}$ B sends update to C: "A via me = cost 3" C updates: $\text{dist}(C,A) = 3+1 = 4$ C tells B: "A cost 4" → B updates to 5 B tells C: "A cost 5" → C updates to 6 ... counts up slowly to infinity (RIP stops at 16 = "unreachable")

Problem: False routes persist for long time → routing loops → wasted bandwidth

Solution	How it works	Limitation
Split Horizon	Do not send route back to the neighbor you learned it from	Doesn't fix routing loops
Poison Reverse	Send route back but with metric= ∞ (explicitly advertise ∞)	Extraneous bandwidth for ∞ updates
Hold-down Timer	After failure, ignore better routes for hold-down period	Slows convergence
Max hop count	RIP: 16=unreachable; limits damage	Still can take 16 iterations
Triggered Updates	Send update immediately when change detected (don't wait 30s)	Reduces but doesn't eliminate loops

Topic 4 – Link State Routing + Dijkstra

Complete Algorithm + Step-by-Step
Table

Link State Routing – Core Concept:

Link State mein har router **poori network ki complete map** jaanta hai. Ye 5 steps follow karta hai: neighbor discovery → LSA generation → flooding → link-state database → Dijkstra's algorithm. OSPF (Open Shortest Path First) iska real-world implementation hai.

LSA Flooding – Kaise kaam karta hai?

1

Hello Packets

Router boot hone par Hello packets bhejta hai → Neighbors discover hote hain + cost measure hoti hai

2

LSA Generation

Router ek LSA packet banata hai: [Router ID, Seq No, Links: {neighbor, cost}]

3

Reliable Flooding

LSA poore network mein flood hoti hai. Har router forward karta hai (duplicate check se)

4

LSDB Build

Har router apna Link State DataBase update karta hai — full network topology visible

5

Dijkstra Run

Har router independently Dijkstra run karta hai apna shortest path tree banane ke liye

Dijkstra's Algorithm – Complete Step-by-Step

Network Graph: A [2] [5] [1] [B] [E] [4] [1] [1] [D] [F] [C] [1] [2] [2] (done) Edges: A-B=2, A-D=4, A-E=5, B-F=1, B-D=3, D-E=3, D-C=2, E-C=1, F-D=2

Dijkstra Algorithm Steps (Source = A):

Initialize: Visited={}, dist[A]=0, dist[all others]=∞ Priority Queue (min-heap): [(0,A)]

Iteration	Node Visited	Dist[A]	Dist[B]	Dist[C]	Dist[D]	Dist[E]	Dist[F]	Updates Made
Init	–	0	∞	∞	∞	∞	∞	Set source=0
1	A (cost=0)	0	2	∞	4	5	∞	B←2(A), D←4(A), E←5(A)
2	B (cost=2)	0	2	∞	4	5	3	F←3(B), D: 2+3=5>4 no change
3	D (cost=4)	0	2	6	4	5	3	C←6(D), E: 4+3=7>5 no change

4	F (cost=3)	0	2	6	4	5	3	D: 3+2=5>4 no change
5	E (cost=5)	0	2	6	4	5	3	C: 5+1=6 same
6	C (cost=6)	0	2	6	4	5	3	No updates

✓ Final Answer

Final Shortest Paths from A: $A \rightarrow B = 2$ (direct) $A \rightarrow C = 6$ ($A \rightarrow D \rightarrow C$) $A \rightarrow D = 4$ (direct) $A \rightarrow E = 5$ (direct) $A \rightarrow F = 3$ ($A \rightarrow B \rightarrow F$) **Shortest Path Tree:** A ■■2■■B ■■1■■F | 4■■D ■■2■■C | 5■■E

Topic 5 – ICMP

All Message Types + ping +
traceroute Working

ICMP – Internet Control Message Protocol:

ICMP ko IP ka "error reporting system" bolte hain. IP ek unreliable protocol hai — koi guarantee nahi ki packet pahunche. Jab kuch galat ho, ICMP source ko inform karta hai. ICMP messages IP datagrams ke andar encapsulate hote hain.

All ICMP Message Types – Complete Table

Type	Code	Message Name	When Sent	Common Use
0	0	Echo Reply	In response to Echo Request	ping ka response
3	0	Net Unreachable	Network not reachable	Routing table no entry
3	1	Host Unreachable	Host not responding	ARP failed/host down
3	2	Protocol Unreachable	Protocol not supported on destination	Wrong protocol
3	3	Port Unreachable	No service on that port	UDP port scan result
3	4	Fragmentation Needed + DF set	Packet too big, DF=1	Path MTU discovery
4	0	Source Quench	Router buffer full — slow down	Deprecated congestion signal
5	0	Redirect (Network)	Use different gateway for this network	Route optimization
5	1	Redirect (Host)	Use different gateway for this host	Route optimization
8	0	Echo Request	Sent by ping command	Testing host reachability
11	0	Time Exceeded (TTL=0 in transit)	Router drops packet with TTL=0	Traceroute uses this
11	1	Time Exceeded (reassembly timeout)	All fragments didn't arrive in time	Fragmentation failure
12	0	Parameter Problem	Bad IP header field	Protocol implementation bug

How ping Works – Step by Step

Command: ping 8.8.8.8
Step 1: Your PC creates ICMP Echo Request (Type=8, Code=0) → Includes: sequence number, identifier, timestamp, payload data
Step 2: Echo Request sent as IP packet (protocol=1 for ICMP) → Destination = 8.8.8.8 (Google DNS)
Step 3: 8.8.8.8 receives the request → creates Echo Reply (Type=0, Code=0) → Copies same sequence number, identifier back
Step 4: Echo Reply returns to your PC
Step 5: Your PC calculates RTT = time_received - time_sent → shows in ms
Output example:
Reply from 8.8.8.8: bytes=32 time=12ms TTL=118 (TTL=118 means 128-10=10 routers traversed if original TTL was 128)
If host unreachable: ICMP Type 3 "Destination Unreachable" returned by router

How traceroute Works – TTL Trick

Command: tracert google.com (Windows) / traceroute google.com (Linux) Core Trick: TTL ko 1 se badhate jaate hain, har bar ek naya router reveal hota hai TTL=1: Packet Router1 pe expire → Router1 ICMP Type 11 (Time Exceeded) bhejta hai → We see Router1's IP + RTT! TTL=2: Packet Router2 pe expire → Router2 Type 11 bhejta hai → We see Router2's IP + RTT! TTL=3: Packet Router3 pe expire → Router3 Type 11 bhejta hai... Continue until: Destination reached → Echo Reply (Type 0) received! Sample output: 1 2ms 1ms 1ms 192.168.1.1 [Your router] 2 10ms 11ms 9ms 10.0.0.1 [ISP gateway] 3 15ms 14ms 16ms 72.14.201.1 [Google's edge] 4 12ms 11ms 12ms 8.8.8.8 [Destination reached!]

Topic 6 – Congestion Control

Leaky Bucket + Token Bucket Deep
+ Numericals

Congestion kab aur kyon hoti hai?

Network mein jab **input rate > output capacity** ho jaaye, router ke buffers full ho jaate hain. Phir naye packets drop hone lagte hain. Packet drop se TCP retransmit karta hai → aur zyada traffic → **congestion collapse** ho sakta hai.

Cause	Why it leads to congestion
Slow output links	High-speed input → slow output → buffers fill up quickly
Multiple inputs, one output	Many fast senders → one slow output link → bottleneck
Bursty traffic	Applications send data in bursts → temporary overflow
Insufficient buffer	Small buffer → drops even moderate bursts
No flow control	Sender doesn't adapt to receiver/network speed

Leaky Bucket – Complete Deep Explanation

Analogy: Ek bucket mein paani daalo (bursty input). Bucket ke neeche ek chhota hole hai jisse paani constant rate pe trickle karta hai. Chahe upar se paani tezi se aaye ya dhire — neeche constant rate pe hi niklega. Bucket overflow → paani (packets) gir jaata hai (drop). **Algorithm:** 1. Bucket has finite capacity B (buffer size) 2. Packets arrive at variable rate (bursty) 3. If bucket has space → accept packet 4. If bucket full → DROP the incoming packet 5. Packets leave bucket at CONSTANT rate r (e.g. 1 packet/ms) **Key property:** Output is ALWAYS at rate r — smooth, no bursts ever pass through **Drawback:** Even if network is completely free, cannot send faster than r

Leaky Bucket Numerical:

Given: Bucket size = 5 packets, Output rate $r = 1$ packet/ms Packet arrivals: $t=0$: 3 packets, $t=1$: 4 packets, $t=6$: 2 packets Timeline: $t=0$: 3 packets arrive → bucket has 0 → accept all 3. Bucket=3 $t=0 \rightarrow 1$: 1 packet transmitted. Bucket=2 $t=1$: 4 packets arrive. Bucket=2, space=3 → accept 3, DROP 1. Bucket=5 $t=1 \rightarrow 6$: 5 packets transmitted (one per ms). Bucket=0 $t=6$: 2 packets arrive → accept all 2. Bucket=2 $t=6 \rightarrow 8$: 2 packets transmitted. Bucket=0 **Total packets: 9 arrived, 1 dropped, 8 transmitted**

Token Bucket – Complete Deep Explanation

Analogy: Ek bucket hai jisme tokens (tickets) hain. Network mein data bhejne ke liye token chahiye — jaise metro mein ticket chahiye. Tokens constant rate pe add hote hain (ek token per second). Aapke paas stored tokens hain → burst mein bhej sakte hain. Tokens khatam → wait karo ya drop karo. **Algorithm:** 1. Token bucket capacity = b tokens 2. Tokens added at rate r tokens/second 3. If bucket full → new tokens DISCARDED (not packets!) 4. To send packet of size L bits: need $L/\text{packet_unit}$ tokens 5. Tokens available → send immediately 6. No tokens → wait (or drop in some implementations) **Key property:** Max burst = b tokens used at once + network rate Maximum burst size = $b + r \times T$ in time T seconds **Advantage:** Stored tokens allow bursts — flexible unlike leaky bucket

Token Bucket Numerical:

Given: Token rate $r = 2$ tokens/ms, Bucket capacity $b = 8$ tokens Packet size = 1 token each. What is maximum burst at $t=0$ if bucket was full? $t=0$: Bucket = 8 (full). Can send 8 packets immediately as burst! After burst: Bucket = 0 $t=1\text{ms}$: 2 new tokens added → send 2 packets $t=2\text{ms}$: 2 new tokens → send 2 packets ... steady state: 2 packets/ms = r **Maximum burst size (if idle for $T=10\text{ms}$ before):** Tokens accumulated = $\min(r \times T, b) = \min(2 \times 10, 8) = \min(20, 8) = 8$ (Bucket capped at 8) Plus new tokens during burst transmission = $r \times (\text{burst_time})$ Max burst = $b = 8$ packets at once (limited by bucket size) **After burst, sustained rate = $r = 2$ packets/ms**

Comparison Point	Leaky Bucket	Token Bucket
Output rate	Strictly constant = r	Variable, up to $r + \text{burst}$
Bursty input allowed?	Yes (up to bucket size)	Yes (uses stored tokens)
Bursty OUTPUT allowed?	NO — always smoothed	YES — stored tokens = burst
Dropped entity	Incoming packets (overflow)	Tokens (if bucket full, not packets)
Flexibility	Rigid — no burst ever	Flexible — burst allowed
Best metric	Bandwidth enforcement	Traffic policing + burst
Real-world use	ATM networks	Internet QoS, ISP rate limiting
Network response	Smooth traffic always	Efficient for bursty apps

Network Layer (IP) logical address use karta hai. Data Link Layer (Ethernet) physical MAC address use karta hai. Jab IP packet frame mein encapsulate karna ho, MAC address chahiye. ARP is "missing link" provide karta hai — IP → MAC mapping.

Scenario: PC-A (IP: 192.168.1.10, MAC: AA:11) wants to send data to PC-B (IP: 192.168.1.20) PC-A does not know PC-B's MAC address.

Step 1 – ARP Cache Check ■ ■ PC-A checks ARP Cache: "Do I have 192.168.1.20 → MAC entry?" ■ ■ → NO → proceed to ARP Request

Step 2 – ARP Request (BROADCAST) ■ ■ PC-A broadcasts: Dst MAC = FF:FF:FF:FF:FF:FF (all devices) ■ ■ Message: "Who has 192.168.1.20? Tell 192.168.1.10 (AA:11)" ■ ■ ALL devices on subnet receive this frame

Step 3 – ARP Reply (UNICAST) ■ ■ PC-B (192.168.1.20) responds: "I have 192.168.1.20, my MAC ■ ■ is BB:22" → sends directly (unicast) to PC-A (AA:11) ■ ■ PC-A caches: 192.168.1.20 → BB:22 (TTL: 2-20 minutes)

Step 4 – Data Transfer ■ ■ PC-A now knows BB:22 → sends Ethernet frame directly to PC-B

Field	Size	Value / Meaning
Hardware Type	2 bytes	1 = Ethernet
Protocol Type	2 bytes	0x0800 = IPv4
Hardware Address Length	1 byte	6 (MAC = 6 bytes)
Protocol Address Length	1 byte	4 (IP = 4 bytes)
Operation	2 bytes	1 = ARP Request; 2 = ARP Reply
Sender MAC Address	6 bytes	Sender's MAC
Sender IP Address	4 bytes	Sender's IP
Target MAC Address	6 bytes	FF:FF:FF:FF:FF:FF (request) / actual (reply)

Target IP Address	4 bytes	Target IP being queried
-------------------	---------	-------------------------

DHCP – DORA Process Complete

DORA = Discover → Offer → Request → Acknowledge

D – DHCP Discover (Broadcast) New device boots → no IP yet → src=0.0.0.0, dst=255.255.255.255 "Is there a DHCP server? I need an IP address!" Port: src=68 (client), dst=67 (server)

O – DHCP Offer (Broadcast or Unicast) DHCP Server responds: "I have an IP for you: 192.168.1.100" Offers: IP + subnet mask + gateway + DNS + lease time

R – DHCP Request (Broadcast) Client broadcasts: "I want the IP from server 192.168.1.1" (Broadcast so other DHCP servers know this client chose one server)

A – DHCP Acknowledge (Broadcast or Unicast) Server confirms: "192.168.1.100 is yours for 24 hours (lease)" Client configures IP, mask, gateway, DNS

Lease Renewal: At 50% of lease time, client tries to renew (DHCP Request) **Lease Expiry:** Client must stop using IP and start DORA again

Topic 8 – IP Fragmentation

Header Fields + Complete
Numerical

Why Fragmentation?

Different network technologies mein alag **MTU (Maximum Transmission Unit)** hoti hai. Ethernet MTU = 1500 bytes. WiFi MTU = 2304 bytes. PPP MTU = varies. Jab IP datagram MTU se bada ho, router usse tod deta hai — **fragmentation**. Destination pe reassembly hoti hai (routers reassemble nahi karte — sirf destination).

IP Header – Fragmentation-related Fields

IP Header Field	Bits	Role in Fragmentation
Identification	16 bits	Same value for all fragments of same datagram (links fragments)
Don't Fragment (DF)	1 bit	DF=1: Don't fragment; if too big → ICMP Type 3 Code 4 returned
More Fragments (MF)	1 bit	MF=1: More fragments coming; MF=0: This is the LAST fragment
Fragment Offset	13 bits	Starting byte position ÷ 8 (in 8-byte units). First fragment = 0
Total Length	16 bits	Total size of THIS fragment (header + data)
TTL	8 bits	Decrement at each router; 0 → drop + ICMP Time Exceeded

Fragmentation Numerical – Complete Solution ■

Given: Original datagram total size = 4000 bytes IP Header = 20 bytes MTU of outgoing link = 1500 bytes

Step 1: Data payload in original Original data = 4000 - 20 = 3980 bytes **Step 2: Max data per fragment**

Max data/fragment = MTU - IP header = 1500 - 20 = 1480 bytes Must be multiple of 8 → $1480 \div 8 = 185$ ✓ (exact multiple) **Step 3: Number of fragments** Fragments = $\text{ceil}(3980 / 1480) = \text{ceil}(2.689) = 3$ fragments

Step 4: Fragment sizes Fragment 1: 1480 bytes data (bytes 0–1479) Fragment 2: 1480 bytes data (bytes 1480–2959) Fragment 3: 1020 bytes data (bytes 2960–3979) [3980-2960=1020] **Step 5: Offsets (divide start byte by 8)** Fragment 1 offset = $0 / 8 = 0$ Fragment 2 offset = $1480 / 8 = 185$ Fragment 3 offset = $2960 / 8 = 370$

Fragment	Data Size (bytes)	Total Size (bytes)	ID	MF Flag	Offset	Start Byte
Fragment 1	1480	1500	x (same)	1	0	0
Fragment 2	1480	1500	x (same)	1	185	1480
Fragment 3	1020	1040	x (same)	0	370	2960

Reassembly at Destination: 1. Same Identification value → belong to same datagram 2. Sort by Fragment Offset (0, 185, 370) 3. Last fragment identified by MF=0 4. Assemble: bytes 0-1479 + 1480-2959 + 2960-3979 = 3980 bytes = original data **If any fragment is lost:** Reassembly timer expires → ALL fragments discarded → ICMP Type 11 Code 1 (Reassembly time exceeded) sent to source → Entire datagram must be retransmitted

Topic 9 – IPv4 vs IPv6

All Differences + Address Types + Transition

Why IPv6 needed?

IPv4 mein sirf $2^{32} \approx 4.3$ billion addresses hain. Internet growth ki wajah se addresses khatam ho gaye (IPv4 exhaustion ~2011). NAT (Network Address Translation) se kuch time mila — lekin permanent solution = IPv6 (128-bit = 340 undecillion addresses — practically infinite).

Feature	IPv4	IPv6
Address length	32 bits (4 bytes)	128 bits (16 bytes)
Total addresses	~4.3 billion (2^{32})	~ 3.4×10^{38} (2^{128})
Notation	Dotted decimal: 192.168.1.1	Hex colon: 2001:db8::1
Header size	20–60 bytes (variable due to options)	40 bytes (fixed — faster processing)
Header checksum	Yes (recalculated at each router)	Removed (upper layers handle it)
Options in header	Yes (variable length, slow)	Extension headers (separate, optional)
Fragmentation	By routers AND source	Only by source; routers send ICMP error
NAT required?	Yes (address exhaustion)	No (enough addresses for everything)
IPSec	Optional (add-on)	Mandatory support built-in
Auto-configuration	Requires DHCP	SLAAC (Stateless) + DHCPv6 (Stateful)
Broadcast	Yes (255.255.255.255)	No broadcast → replaced by multicast
Anycast	Not standard	Native support
QoS support	ToS (Type of Service) field	Traffic Class + Flow Label (20 bits)
Mobility	Mobile IP (complex)	Built-in mobility support
Loopback	127.0.0.1	::1
All-nodes multicast	255.255.255.255	FF02::1
Header fields count	12–15 fields	8 fields (simpler)

IPv6 Address Types

Type	Prefix	Scope	Example / Use
Global Unicast	2000::/3	Internet-wide (routable)	2001:db8::/32 — public IPv6 addresses
Link-Local	FE80::/10	Single link only	Auto-assigned; used for neighbor discovery
Unique Local	FC00::/7	Organization-wide	Like private IPv4 (10.x, 192.168.x)
Loopback	::1/128	Self only	Equivalent to IPv4 127.0.0.1

Unspecified	::	Source when no address is specified	Like 0.0.0.0 in IPv4 (DHCP discovery)
Multicast	FF00::/8	Group	FF02::1=all nodes; FF02::2=all routers
Anycast	From unicast space	Nearest interface	CDN, DNS — nearest server responds

IPv6 Address Notation Rules

Full IPv6 address: 2001:0db8:0000:0000:0000:ff00:0042:8329 **Rule 1 – Leading zeros drop:** 0db8 → db8; 0042 → 42 → 2001:db8:0:0:0:ff00:42:8329 **Rule 2 – Consecutive all-zero groups → :: (only ONCE):** 0:0:0 → :: → 2001:db8::ff00:42:8329 **Examples:** ::1 = 0000:0000:0000:0000:0000:0000:0000:0001 (loopback) :: = all zeros (unspecified) 2001:db8::1 = 2001:0db8:0000:0000:0000:0000:0000:0001

Master Revision Sheet – Unit 3

All Formulas + Key Points + Exam Tips

Topic	Formula / Key Point
IP Class ID	A:1-126; B:128-191; C:192-223; D:224-239; E:240-255
Usable Hosts	$2^h - 2$ (h=host bits; -2 for network+broadcast)
Subnet bits s	$2^s \geq \text{required_subnets}$; pick smallest s
New prefix length	original_prefix + s (e.g. /24+3=/27)
Block size	2^s or $256 - \text{last_octet_of_mask}$
Network address	IP AND subnet_mask
Broadcast address	Flip all host bits to 1 in network address
CIDR /n	Count 1-bits in mask; /27=255.255.255.224 (8+8+8+3)
DVR algorithm	Bellman-Ford: $D(x,y) = \min\{\text{cost}(x,v) + D(v,y)\}$ for all neighbors v
DVR convergence	Slow; shares full routing table; shares with neighbors only
Count-to-infinity	Fix: Split Horizon / Poison Reverse / Hold-down / max=16
RIP metric	Hop count; max 15 hops; 16=unreachable; updates every 30s
Dijkstra init	source=0; all others= ∞ ; pick min unvisited; update neighbors
OSPF	Link State; uses Dijkstra; flooding LSA; areas (0=backbone)
ICMP Type 0	Echo Reply → ping response
ICMP Type 3	Destination Unreachable (Code 0=net, 1=host, 3=port)
ICMP Type 8	Echo Request → what ping sends
ICMP Type 11	Time Exceeded (TTL=0) → traceroute uses this
Leaky Bucket	Constant output rate r; overflow=drop; no burst output
Token Bucket	Tokens at rate r; burst allowed using stored b tokens; flexible
Max Token Bucket burst	b tokens (bucket capacity) at any instant
ARP purpose	IP → MAC; broadcast request; unicast reply; cache result
DHCP DORA	Discover(bcast)→Offer(server)→Request(bcast)→ACK(server)
IP Frag offset	start_byte / 8 (8-byte units in IP header field)
IP Frag max data	MTU - 20 (IP header); must be multiple of 8
MF flag	More Fragments: 1=more coming; 0=last fragment
DF flag	Don't Fragment: 1=don't fragment; ICMP error if too big
IPv4 vs IPv6	32-bit vs 128-bit; 4.3B vs 340 undecillion addrs; broadcast vs multicast
IPv6 loopback	::1 (equivalent to 127.0.0.1 in IPv4)
IPv6 SLAAC	Stateless Address AutoConfig: device creates own IPv6 from MAC + prefix

■ EXAM STRATEGY – Unit 3

■ TOP 6 EXAM QUESTIONS – Unit 3 (Must Prepare These):

1. Subnetting Numerical (10M – GUARANTEED):

200.1.2.0 → 5 subnets: s=3, mask=/27=255.255.255.224, block=32, hosts=30

Write full table: subnet, network addr, first host, last host, broadcast

2. DVR Numerical (10M):

Given topology → write initial tables → iterate 2-3 times → show convergence

3. Dijkstra's Algorithm (10M):

Given graph → step-by-step table (Visited, dist[each node], updates) → shortest path tree

4. Leaky Bucket vs Token Bucket (10M):

Both algorithms with diagram → comparison table → numerical example

5. ICMP Messages (5M):

Table of Type + Code + Message + When sent → ping working → traceroute working

6. IP Fragmentation (5M):

4000B datagram, MTU=1500 → 3 fragments: sizes, offsets (0,185,370), MF flags